

Laporan Kemajuan Proyek Pembelajaran Mesin

S1 Sains Data, Fakultas Matematika Ilmu Pengetahuan dan IPA
Universitas Negeri Surabaya

Kelas	2024-C
Ketua	Chaesar Giveson (24031554058)
Anggota 1	Sasmita Kusuma Jati (24031554052)
Anggota 2	Nagatan Alief Putra Silahen (24031554086)
URL Github	https://github.com/KsmaJatt/Projek-ML

Informasi Umum Proyek

Judul	Klasifikasi Penyakit Tanaman Menggunakan Perbandingan CNN, EfficientNetB0, dan Hybrid EfficientNetB0-XGBoost pada Dataset PlantVillage untuk Mendukung Ketahanan Pangan
Topik	Pangan
Pilihan Rumusan Masalah	1.3 Dampak Perubahan Iklim dan Rendahnya Adopsi Pertanian Berkelanjutan
Revisi?	Ya / Tidak

Informasi Detil Dataset

Sumber dataset	Tuliskan bagaimana proses akuisisi data untuk penyusunan dataset, atau URL sumber dataset (https://www.kaggle.com/datasets/emmarex/plantdisease)
Deskripsi awal fitur dataset	1. image Atribut ini merupakan data utama dalam dataset berupa citra daun tanaman yang diambil dalam format RGB. Semua foto direpresentasikan dalam format RGB dan dinormalisasi untuk menyesuaikan proses deep learning. Tipe data atribut ini adalah array tiga dimensi ($H \times W \times 3$) yang merepresentasikan tinggi, lebar, dan tiga kanal warna citra. 2. image_id Atribut ini merupakan identifikasi unik untuk setiap file gambar dalam dataset, biasanya berupa nama file seperti <code>0a1e8d5e-cc84.jpg</code> yang ditetapkan secara otomatis saat dataset dikompilasi. Tipe data atribut ini adalah String atau Categorical yang berfungsi sebagai kunci primer untuk membedakan satu sampel citra dengan sampel lainnya. 3. label / class Setiap direktori dalam dataset menyertakan label spesies dan status kesehatan citra, misalnya direktori bernama <code>Tomato_Late_blight</code> hanya berisi gambar daun tomat dengan gejala penyakit hawar daun, sedangkan <code>Apple_Healthy</code> berisi gambar daun apel yang sehat. Atribut ini bertipe Categorical dengan total 38 kelas unik yang menjadi target klasifikasi dalam penelitian. 4. plant_species Dataset ini mencakup 14 spesies tanaman: Apple, Blueberry, Cherry, Corn, Grape, Orange, Peach, Bell Pepper, Potato, Raspberry, Soybean, Squash, Strawberry, dan Tomato. Atribut ini bertipe Categorical dengan 14 kategori dan

	berfungsi sebagai informasi taksonomi dasar sebelum proses klasifikasi penyakit dilakukan. 5. disease_name Atribut ini menyimpan nama spesifik penyakit yang menginfeksi daun tanaman, atau keterangan "Healthy" apabila daun tidak menunjukkan gejala infeksi apapun. Tipe data atribut ini adalah Categorical yang nilainya diturunkan langsung dari penamaan folder dalam struktur direktori dataset. 6. disease_type Dataset ini mencakup 17 penyakit jamur (fungal), 4 penyakit bakteri (bacterial), 2 penyakit mold (oomycete), 2 penyakit virus (viral), dan 1 penyakit yang disebabkan oleh tungau (mite). Atribut ini bertipe Categorical dan penting untuk analisis distribusi jenis patogen dalam kerangka penelitian ketahanan pangan. 7. health_status Atribut ini merepresentasikan kondisi daun secara biner, yaitu apakah daun tergolong sehat (Healthy) atau berpenyakit (Diseased), dan diturunkan dari atribut label. Tipe datanya adalah Binary atau Categorical yang berguna untuk analisis ketidakseimbangan kelas sebelum pelatihan model dilakukan. 8. image_resolution Dataset menggunakan resolusi gambar seragam sebesar 224×224 piksel untuk memastikan konsistensi dimensi input pada seluruh arsitektur deep learning yang digunakan. Tipe data atribut ini adalah Integer yang merepresentasikan jumlah piksel pada sumbu horizontal dan vertikal citra. 9. color_channel Atribut ini merepresentasikan tiga saluran warna pada setiap citra, yaitu Red (R), Green (G), dan Blue (B), dengan nilai intensitas piksel berkisar antara 0 hingga 255 per kanal. Tipe data atribut ini adalah Integer yang menjadi dasar ekstraksi fitur visual oleh lapisan konvolusi pada arsitektur CNN. 10. background_type PlantVillage menggunakan lingkungan terkontrol untuk pengambilan gambar, di mana setiap gambar diambil dengan latar belakang bersih, sudut terkontrol, dan arah kamera yang konsisten. Atribut ini bertipe Categorical dan perlu diperhatikan dalam konteks generalisasi model, mengingat kondisi lapangan nyata umumnya tidak memiliki latar belakang yang terkontrol.
Detil-detil langkah pre-processing	1. Resize Gambar Semua gambar diresize ke ukuran seragam 224×224 piksel agar konsisten sebagai input model. Alasan pemilihan 224×224: ukuran standar untuk arsitektur deep learning berbasis ImageNet, termasuk EfficientNetB0. 2. Normalisasi Nilai Piksel Nilai piksel diubah dari rentang 0–255 menjadi 0–1. Ini membantu gradient descent konvergen lebih stabil. EfficientNetB0 & Hybrid: Menggunakan preprocess_input bawaan EfficientNetB0 yang menangani normalisasi sesuai distribusi data ImageNet (mean dan std per channel RGB). Tidak pakai rescale manual. 3. Train-Validation Split Dataset dibagi dengan rasio 80:20 menggunakan parameter validation_split=0.2 di ImageDataGenerator. Total gambar : 20.638 Train : 16.516 gambar (80%) Validasi : 4.122 gambar (20%) 4. Label Encoding Label kelas diambil otomatis dari nama folder di struktur direktori dataset. Pepper_bell_Bacterial_spot → 0 Pepper_bell_healthy → 1

	Potato___Early_blight → 2							
	...							
	Tomato_healthy → 14							
	5. Augmentasi Data							
	Augmentasi hanya diterapkan pada data training , tidak pada data validasi, untuk menambah variasi dan mencegah overfitting:							
	<i>Tabel 1.1 Parameter Augmentasi Data</i> <table> <tr> <th>Teknik</th><th>Parameter</th></tr> <tr> <td>Rotasi</td><td>rotation_range=20 (±20 derajat)</td></tr> <tr> <td>Zoom</td><td>zoom_range=0.2 (±20%)</td></tr> <tr> <td>Horizontal Flip</td><td>horizontal_flip=True</td></tr> </table>	Teknik	Parameter	Rotasi	rotation_range=20 (±20 derajat)	Zoom	zoom_range=0.2 (±20%)	Horizontal Flip
Teknik	Parameter							
Rotasi	rotation_range=20 (±20 derajat)							
Zoom	zoom_range=0.2 (±20%)							
Horizontal Flip	horizontal_flip=True							
	6. Batching							
	Data dimuat dalam batch untuk efisiensi memori dan training: batch_size = 32							
	Artinya setiap step training, model menerima 32 gambar sekaligus.							

Kemajuan Pengerjaan

Teknik yang digunakan

Pada proyek ini digunakan tiga pendekatan utama untuk klasifikasi penyakit tanaman berbasis citra daun, yaitu CNN sederhana sebagai baseline, transfer learning menggunakan EfficientNetB0, dan model hybrid EfficientNetB0-XGBoost. Ketiga pendekatan dibandingkan untuk mengetahui model yang paling efektif dalam menangani klasifikasi multi-class pada dataset PlantVillage yang memiliki ketidakseimbangan jumlah data antar kelas.

1. CNN Sederhana (Baseline)

CNN digunakan sebagai model baseline untuk memahami performa arsitektur deep learning konvensional tanpa pretrained model. Arsitektur CNN terdiri dari tiga blok convolution dan max pooling.

Arsitektur CNN

Tabel 2.1 susunan layer CNN

Layer	Konfigurasi
Conv2D 32	32 filter, kernel 3×3, aktivasi ReLU
MaxPooling2D	Pool size 2×2
Conv2D 64	64 filter, kernel 3×3, aktivasi ReLU
MaxPooling2D	Pool Size 2x2
Conv2D 128	128 filter, kernel 3×3, aktivasi ReLU
MaxPooling2D	Pool size 2x2

Flatten	Mengubah feature map menjadi vektor
Dropout	0.5
Dense	256 neuron, aktivasi ReLU
Output Dense	Softmax, 15 kelas

CNN bekerja menggunakan operasi konvolusi untuk mengekstraksi fitur visual seperti tepi daun, tekstur bercak penyakit, pola warna, dan perubahan permukaan daun.

Operasi Konvolusi

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

Di mana:

- I = citra input
- K = kernel/filter
- (i, j) = posisi piksel output

Layer pertama menggunakan 32 filter ukuran 3×3 untuk mendeteksi fitur dasar pada gambar daun.

Fungsi Aktivasi ReLU

$$\text{ReLU}(x) = \max(0, x)$$

ReLU digunakan untuk mempertahankan fitur penting dan menghilangkan aktivasi negatif yang dianggap tidak relevan.

Max Pooling

$$\text{MaxPool}(i, j) = \max_{(m, n) \in \text{window}} I(i + m, j + n)$$

Max pooling digunakan untuk mengurangi dimensi feature map sehingga komputasi lebih efisien dan model lebih tahan terhadap translasi kecil pada gambar.

Dropout

Dropout sebesar 0.5 diterapkan sebelum fully connected layer untuk mengurangi overfitting dengan cara menonaktifkan sebagian neuron secara acak saat training.

Softmax Output

$$P(y = k | x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Softmax mengubah output akhir menjadi probabilitas untuk masing-masing kelas penyakit tanaman.

Optimizer dan Loss Function

- *Optimizer: Adam*
- *Loss Function: Categorical Crossentropy*
- *Metrics: Accuracy*

Optimizer Adam dipilih karena mampu melakukan adaptive learning rate sehingga proses training lebih stabil.

2. Transfer Learning EfficientNetB0

EfficientNetB0 digunakan karena memiliki performa tinggi dengan jumlah parameter relatif kecil dibanding CNN konvensional. Model ini menggunakan pretrained weight dari ImageNet sehingga mampu memanfaatkan fitur visual yang telah dipelajari sebelumnya.

Compound Scaling

EfficientNet menggunakan metode compound scaling yang menyeimbangkan scaling depth, width, dan resolution.

$$d = \alpha^\Phi, \quad w = \beta^\Phi, \quad r = \gamma^\Phi$$

Dengan constraint:

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

Pada EfficientNetB0 digunakan input berukuran 224×224 piksel.

Arsitektur Transfer Learning

- *EfficientNetB0 pretrained ImageNet (include_top=False)*
- *GlobalAveragePooling2D*
- *Dropout 0.3*
- *Dense 256 neuron + ReLU*
- *Dropout 0.3*
- *Dense Softmax 15 kelas*

Global Average Pooling

$$GAP(k) = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W f_k(i, j)$$

GlobalAveragePooling2D digunakan untuk mereduksi feature map menjadi vektor fitur tanpa menambahkan terlalu banyak parameter seperti pada fully connected layer biasa.

Strategi Fine-Tuning

Training dilakukan dalam dua fase:

Fase 1

- Seluruh layer EfficientNet dibekukan (frozen)
- Hanya classifier head yang dilatih
- Learning rate = $1e-3$

Fase 2

- 20 layer terakhir EfficientNet dibuka kembali (unfreeze)
- Fine-tuning dilakukan dengan learning rate lebih kecil
- Learning rate = $1e-4$

Pendekatan ini bertujuan menjaga fitur pretrained ImageNet tetap stabil sambil menyesuaikan model terhadap karakteristik dataset daun tanaman.

Loss Function

$$\mathcal{L} = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

Categorical crossentropy digunakan karena kasus klasifikasi terdiri dari banyak kelas.

3. Hybrid EfficientNetB0 + XGBoost

Pendekatan hybrid digunakan dengan memanfaatkan EfficientNetB0 sebagai feature extractor dan XGBoost sebagai classifier akhir.

Tahap 1: Feature Extraction

Feature vector diekstraksi dari layer sebelum classifier akhir EfficientNetB0 menggunakan output GlobalAveragePooling2D.

Hasil ekstraksi menghasilkan feature vector berdimensi 1280 untuk setiap gambar.

Tahap 2: Klasifikasi Menggunakan XGBoost

XGBoost membangun ensemble decision tree secara bertahap untuk memperbaiki error model sebelumnya.

$$\widehat{y}_i^{(t)} = \widehat{y}_i^{(t-1)} + f_t(x_i)$$

Fungsi objektif:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l\left(y_i, \widehat{y_i^{(t-1)}} + f_t(x_i)\right) + \Omega(f_t)$$

Dengan regularisasi:

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

Parameter XGBoost yang digunakan:

- `n_estimators = 200`
- `max_depth = 6`
- `learning_rate = 0.1`
- `eval_metric = mlogloss`

XGBoost dipilih karena mampu menangani feature vector berdimensi tinggi dengan baik serta memiliki regularisasi bawaan untuk mengurangi overfitting.

4. Data Augmentation

Untuk meningkatkan generalisasi model dan mengurangi overfitting, diterapkan augmentasi data pada training set.

Parameter augmentasi:

- `rotation_range = 40`
- `zoom_range = 0.3`
- `horizontal_flip = True`
- `vertical_flip = True`
- `brightness_range = [0.7, 1.3]`
- `shear_range = 0.2`
- `width_shift_range = 0.1`
- `height_shift_range = 0.1`

Augmentasi membantu model mengenali variasi posisi, pencahayaan, dan orientasi daun.

5. Callback Training

Untuk menjaga stabilitas proses training digunakan beberapa callback:

EarlyStopping

Training dihentikan otomatis ketika performa validasi tidak meningkat selama beberapa epoch untuk mencegah overfitting.

Parameter:

- `patience = 5`
- `restore_best_weights = True`

ReduceLROnPlateau

Learning rate akan diperkecil secara otomatis ketika validasi stagnan.

Parameter:

- *factor = 0.2*
- *patience = 3*

ModelCheckpoint

Model terbaik selama training disimpan otomatis berdasarkan nilai validation accuracy tertinggi.

6. Metrik Evaluasi

Evaluasi model dilakukan menggunakan confusion matrix dan beberapa metrik klasifikasi multi-class.

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

$$Precision_k = \frac{TP_k}{TP_k + FP_k}$$

Recall

$$Recall_k = \frac{TP_k}{TP_k + FN_k}$$

F1-Score

$$F1_k = 2 \cdot \frac{Precision_k \times Recall_k}{Precision_k + Recall_k}$$

Digunakan macro average dan weighted average untuk mengevaluasi performa pada dataset imbalance.

Confusion Matrix

Confusion matrix digunakan untuk melihat distribusi prediksi benar dan salah pada masing-masing kelas penyakit tanaman sehingga dapat dianalisis kelas mana yang masih sulit diprediksi model.

7. Framework dan Library

	Framework dan library yang digunakan dalam proyek ini antara lain: • TensorFlow/Keras digunakan untuk membangun, melatih, dan mengevaluasi model deep learning seperti CNN dan EfficientNetB0. • EfficientNetB0 digunakan sebagai arsitektur transfer learning berbasis pretrained ImageNet untuk ekstraksi fitur citra daun. • XGBoost digunakan sebagai classifier pada pendekatan hybrid EfficientNetB0-XGBoost. • Scikit-learn digunakan untuk preprocessing data, perhitungan class weight, confusion matrix, serta evaluasi metrik klasifikasi seperti accuracy, precision, recall, dan F1-score. • NumPy digunakan untuk operasi numerik dan manipulasi array multidimensi. • Pandas digunakan untuk pengolahan dan analisis data tabular. • Matplotlib dan Seaborn digunakan untuk visualisasi data, confusion matrix, serta grafik training dan validation.
Nilai metrik terakhir	Ketiga model telah selesai dilatih dan dievaluasi menggunakan data validasi sebanyak 4.122 sampel yang mencakup 15 kelas penyakit tanaman. Berikut adalah uraian nilai metrik yang diperoleh dari masing-masing model. • Model CNN Sederhana menghasilkan accuracy sebesar 0.9384 dengan F1-score macro average 0.93 dan weighted average 0.94. Model ini mampu mengenali sebagian besar kelas dengan baik, namun menunjukkan kelemahan pada kelas minoritas. Kelas Potato__healthy hanya menghasilkan F1-score 0.79 karena jumlah sampelnya sangat sedikit (30 sampel), sehingga model kesulitan mempelajari pola kelas tersebut secara memadai. Dari confusion matrix, kesalahan terbesar terjadi pada Tomato_Late_blight yang salah diprediksi sebagai Potato__Late_blight sebanyak 24 sampel, serta Tomato_Septoria_leaf_spot yang salah diprediksi sebagai Potato__Early_blight sebanyak 15 sampel. • Model EfficientNetB0 menghasilkan accuracy sebesar 0.9850 dengan F1-score macro average 0.99 dan weighted average 0.98, menjadikannya model dengan performa terbaik dalam penelitian ini. Hampir seluruh kelas mencapai F1-score di atas 0.98, termasuk kelas Potato__healthy yang sebelumnya bermasalah pada CNN kini berhasil mencapai F1-score sempurna 1.00. Satu-satunya kelemahan yang tersisa adalah pada pasangan kelas Tomato_Spider_mites_Two_spotted_spider_mite dan Tomato__Target_Spot yang saling salah prediksi sebanyak 12 sampel, mengindikasikan kemiripan visual yang tinggi antara kedua penyakit tersebut. • Model Hybrid EfficientNetB0 + XGBoost menghasilkan accuracy sebesar 0.9767 dengan F1-score macro average 0.97 dan weighted average 0.98. Model ini menunjukkan performa yang sangat kompetitif, namun sedikit di bawah EfficientNetB0 murni. Penurunan performa ini terjadi karena proses ekstraksi fitur yang dipisahkan dari proses klasifikasi menyebabkan sebagian informasi gradien yang diperoleh selama fine-tuning tidak dapat dimanfaatkan sepenuhnya oleh XGBoost. Kesalahan terbesar ditemukan pada kelas Tomato__Target_Spot yang salah diprediksi ke beberapa kelas lain, serta pasangan Tomato_Spider_mites ↔ Tomato__Target_Spot yang konsisten sulit dibedakan di ketiga model. Secara keseluruhan, ketiga model menunjukkan performa yang baik dengan ranking sebagai berikut: Tabel 3.1 Performa Model

	<i>Peringkat</i>	<i>Model</i>	<i>Accuracy</i>	<i>F1 Macro</i>	<i>F1 Weighted</i>
	1	<i>EfficientNetB0</i>	<i>0.9850</i>	<i>0.99</i>	<i>0.98</i>
	2	<i>Hybrid EffNet+XGBoost</i>	<i>0.9767</i>	<i>0.97</i>	<i>0.98</i>
	3	<i>CNN Sederhana</i>	<i>0.9384</i>	<i>0.93</i>	<i>0.94</i>

Visualisasi grafik
latih dan validasi

CNN Sederhana

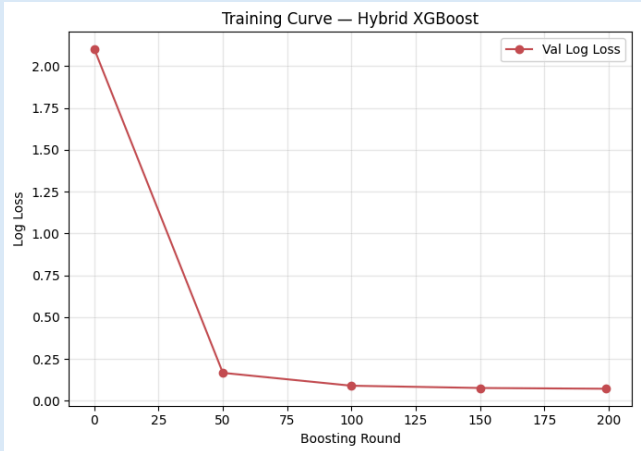
Gambar. 4.1 Curva Latih CNN Sederhana

Berdasarkan kurva pelatihan CNN sederhana, training accuracy menunjukkan peningkatan yang konsisten dari 0.60 pada epoch pertama hingga mencapai 0.97 pada epoch terakhir. Namun, validation accuracy cenderung stagnan di kisaran 0.88–0.89 mulai epoch ke-7 dan tidak mengikuti kenaikan training accuracy. Pola serupa juga terlihat pada kurva loss, di mana training loss terus menurun secara stabil, sementara validation loss mulai berfluktuasi dan menunjukkan tren kenaikan di epoch-epoch akhir. Kondisi ini mengindikasikan terjadinya overfitting ringan, yaitu model terlalu menyesuaikan diri dengan data latih sehingga kemampuan generalisasinya terhadap data baru menjadi terbatas. Hal ini merupakan keterbatasan yang umum terjadi pada arsitektur CNN yang dilatih dari awal tanpa memanfaatkan pretrained weights.

EfficientNetB0

Gambar. 4.2 Curva Latih EfficientNetB0 Phase 1 + Fine Tune

Kurva pelatihan EfficientNetB0 menampilkan dua fase pelatihan yang dibatasi oleh garis merah putus-putus sebagai penanda dimulainya proses fine-tuning pada epoch ke-11. Pada Phase 1 (epoch 1–10), validation accuracy sudah mencapai ~0.94 sejak awal pelatihan meskipun training accuracy masih tertinggal di bawahnya. Pola ini merupakan karakteristik khas transfer learning, di mana pretrained weights dari ImageNet memungkinkan model langsung menghasilkan representasi fitur yang baik sejak iterasi pertama. Setelah fine-tuning dimulai pada Phase 2, training accuracy mengalami lonjakan signifikan dan kedua kurva berkonvergensi di nilai ~0.98 pada epoch terakhir. Kurva loss juga menurun secara mulus tanpa fluktuasi yang berarti.

	Tidak ditemukan tanda-tanda overfitting, sehingga kurva ini menggambarkan proses pelatihan yang ideal dan stabil. Hybrid EfficientNetB0 + XGBoost  Gambar. 4.3 Curva Latih Hybrid EfficientNetB0 + XGBoost Berbeda dengan dua model sebelumnya, XGBoost tidak memiliki konsep epoch dalam proses pelatihannya. Kurva pelatihan yang ditampilkan merupakan nilai validation log loss terhadap jumlah boosting round. Pada round awal, log loss dimulai dari nilai yang tinggi sekitar 2.10 dan mengalami penurunan yang sangat tajam hingga round ke-50 di mana log loss turun drastis ke ~0.15. Setelah itu, penurunan melambat secara bertahap hingga mencapai nilai stabil di sekitar 0.07 pada round ke-150 dan tidak menunjukkan perubahan signifikan hingga round ke-200. Pola ini menunjukkan bahwa model telah konvergen dengan baik dan penambahan jumlah pohon di atas 150 tidak lagi memberikan peningkatan performa yang berarti. Dengan demikian, penggunaan 200 estimator pada penelitian ini sudah memadai untuk menghasilkan klasifikasi yang optimal.
Temuan	<i>Berdasarkan hasil pelatihan dan evaluasi, ditemukan bahwa transfer learning menggunakan EfficientNetB0 memberikan performa terbaik dibandingkan CNN sederhana maupun hybrid EfficientNetB0 + XGBoost. EfficientNetB0 berhasil mencapai accuracy sebesar 98.50% dengan kurva training yang stabil dan minim overfitting. Hal ini menunjukkan bahwa pretrained weights dari ImageNet mampu membantu model mengenali fitur visual penyakit tanaman secara lebih efektif dibandingkan model CNN.</i> <i>CNN sederhana menunjukkan performa paling rendah serta mengalami overfitting ringan selama proses pelatihan. Training accuracy terus meningkat hingga sekitar 97%, namun validation accuracy cenderung stagnan pada kisaran 88–89% sehingga terlihat adanya gap antara data latih dan validasi. Kondisi ini menunjukkan bahwa model terlalu menyesuaikan diri terhadap data latih dan kurang mampu melakukan generalisasi pada data baru, terutama pada kelas dengan jumlah sampel yang sedikit.</i> <i>Model hybrid EfficientNetB0 + XGBoost menghasilkan performa yang kompetitif dengan accuracy sebesar 97.67%, namun masih sedikit di bawah EfficientNetB0 murni. Hal ini diduga karena proses ekstraksi fitur dan klasifikasi dilakukan secara terpisah sehingga optimisasi end-to-end tidak dapat dimanfaatkan secara penuh. Selain itu, ditemukan bahwa beberapa kelas seperti Tomato_Spider_mites_Two_spotted_spider_mite dan Tomato__Target_Spot memiliki kemiripan visual tinggi sehingga konsisten sulit dibedakan oleh ketiga model.</i>

	<i>Distribusi jumlah data yang tidak seimbang antar kelas juga memengaruhi performa model, terutama pada CNN sederhana yang mengalami penurunan performa pada kelas minoritas seperti Potato___healthy. Kelas dengan jumlah sampel sedikit cenderung memiliki nilai F1-score yang lebih rendah dibandingkan kelas mayoritas. Berdasarkan hasil sementara, proses penelitian selanjutnya akan dilanjutkan dengan penanganan class imbalance dan hyperparameter tuning untuk meningkatkan stabilitas serta performa klasifikasi model.</i>
--	--